

Macaroons allow authority to be *delegated* between protection domains with both *attenuation* and *contextual confinement*. For this, each macaroon embeds *caveats* which are predicates that restrict the macaroon’s authority, as well as the context in which it may be successfully used. For example, such caveats may attenuate a macaroon by limiting what objects and what actions it permits, or contextually confine it by requiring additional evidence, such as third-party signatures, or by restricting when, from where, or in what other observable context it may be used. In particular, a macaroon that authorizes service requests may contain caveats that require proof that the requests have been audited and approved by an abuse-detection service, and come from a specific device with a particular authenticated user.

In short, Macaroons allow Cloud services to authorize resource access using efficient, restricted bearer tokens that can be delegated further, and, via embedded caveats, attenuated in scope, subjected to third-party inspection and approval, and confined to be used only in certain contexts. Fundamental to this flexibility of macaroons is a chained construction of nested HMAC values that simultaneously provides key distribution and protects integrity, generalizing earlier work dating back to the Amoeba operating system [3, 25, 51].

II. MOTIVATION AND FOUNDATIONS

Conceptually, macaroons make an assertion about conditional access to a *target service*¹, along these lines: “The bearer may perform a request *as long as* predicates $C_1, \dots, C_n$ hold true in the context of that request.”

In distributed systems, the complexity of such assertions can grow surprisingly quickly, even for simple, practical authorization policies. For example, if, as in Section I, the owner of a private image at *TS* wants to safely email its URL link to the members of a group *G* kept at an unrelated service *A*, macaroons may establish an assertion like the following:

The bearer, client *C*, may perform the request at *TS*
as long as the operation is read,
as long as the object is `privateImage.jpg`,
as long as the user at *C* is logged into service *A*,
as long as that logged-in user is in group *G* at *A*,
as long as the above proofs are recent/fresh enough,
 and, *as long as* the user at *C* didn’t just log out of *A*.

Furthermore, the image may be subject to additional access controls at *TS* (e.g., to track changes of ownership), the request from client *C* may be performed from an unnamed context (e.g., a transient Web-page frame), and the logged-in user’s privacy may need to be protected from *TS*. (Variants of this assertion are the basis for paper’s examples and the end-to-end evaluation in Section VII.)

Due to their flexible, efficient HMAC-based construction, macaroons can enforce such complex assertions despite tight freshness constraints and the need to closely guard privacy.

Macaroons are an instance of what has been termed *claims-based*, *proof-carrying*, or simply *credentials-based authorization*, and is defined precisely in [48, Chapter 9]. Over the last

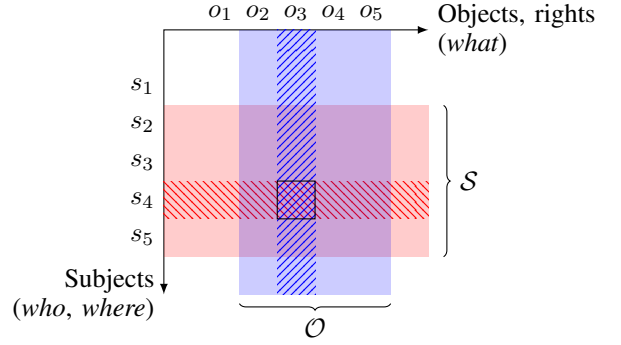


Fig. 1. Macaroons in the Access Control Matrix model [36]. By attenuating a macaroon authorizing $\mathcal{O}$, a derived macaroon may permit access only to o_3 . Even when held by all subjects in $\mathcal{S}$, contextual caveats may confine the use of this derived macaroon to the observable context of principal s_4 .

two decades, several such credentials-based mechanisms have been developed using public-key certificates and the formalism of authorization logic [37, 53]. Of these, many support highly flexible, decentralized authorization for distributed systems, with notable examples including SPKI/SDSI [14], Trust Management [11], active certificates [12], and SecPAL [9]. However—whether due to the cost of their primitives, their need for federation, coordination, or public identities, or the complexity of their implementation—such flexible public-key-certificate mechanisms have seen little use in the Cloud [54].

Instead, near exclusively, authorization in the Cloud is based on a pattern of pure *bearer tokens*: secrets that grant unconditional authority, and are sent directly between protection domains, even on unencrypted channels [21]. Examples of pure bearer tokens include the HTTP cookies sent to identify authorized sessions, the secrets for API keys, OAuth tokens, and SAML or OpenID assertions used at other protocol layers, as well as the “unlisted URLs” that are commonly created and shared via email or chat to provide limited access to private content [28, 50, 54].

Efficiency, wide applicability, and ease-of-adoption are three main reasons for bearer tokens’ popularity in the Cloud. Macaroons share all three of these properties, in their HMAC-based construction. First, macaroons are simple enough to implement, and fast-enough to use in near any environment—even a severely-limited, out-of-date Web browser executing on slow, resource-constrained mobile hardware. Second, macaroons are *bearer credentials* and, so, principals may gain authority simply by gathering macaroons. Third, macaroons still provide new benefits even when unilaterally adopted, e.g., by allowing users to restrict when, and from where, credentials may be used [21].

By supporting *attenuation*, *delegation*, and *contextual confinement*, macaroons have much greater expressiveness than previously existing Cloud bearer tokens. Thus, as shown in Figure 1, a user that holds access rights to the objects in $\mathcal{O}$ can safely pass, via email, a macaroon for the right to access a single object o_4 , limited to the specific recipient s_4 , even if all the parties in $\mathcal{S}$ can read the email. Moreover, as shown in Figure 2, the user can do this without the operational difficulty or latency and bandwidth costs of running an intermediate proxy service, and even without contacting the target service to mint new, restricted credentials.

¹This section shows key terms and new concepts in *italics* when introduced and informally defined; the next two sections and the appendix provide detailed definitions and describe concrete implementations.